

Automated Delivery Scheduling System

Project Problem Statement

Project Overview

RoboWorks operates a fleet of autonomous delivery robots across multiple zones. The company requires a reliable software system to load, validate, and evaluate the feasibility of delivery tasks and robot schedules. Without such a system, it is not possible to determine in advance whether a robot has sufficient battery life, load capacity, or zone access to complete a given set of deliveries.

Problem Statement

Design and implement a modular Python-based delivery scheduling system that reads and validates operational data from structured CSV files, determines the feasibility of delivery tasks and robot schedules by simulating physical constraints, and produces a written feasibility report alongside a visualisation of robot movement over time.

Key Challenges

- Reading and parsing CSV data across four file types: robots, destinations, packages, and tasks, each with different fields and validation rules.
- Validating all records using regular expressions to enforce correct ID formats, zone formats, integer and float representations, and value bounds. Invalid records must be discarded with a descriptive warning.
- Determining task feasibility by checking whether any available robot shares the required zone and has sufficient load capacity for the associated package.
- Simulating schedule execution to track battery consumption, distance travelled, and time elapsed across a sequence of tasks, using a distance matrix to compute travel between destinations.
- Generating a structured feasibility report that summarises task and schedule outcomes, including time, distance, and remaining battery for completed schedules.
- Visualising feasible robot schedules by plotting distance from the origin over time using pandas and matplotlib.

System Requirements

Data Loading and Validation

- Read robots, destinations, packages, tasks, schedules, and distances from CSV files into structured data tables (lists of dictionaries).
- Validate all fields using the re module. Reject and warn on any record that fails format or bounds checks.
- Robot battery must be an integer between 0 and 100. Max load must be a non-negative float. Zone must be uppercase alphabetical characters only.
- Task source, target, and package IDs must reference records that passed validation in their respective files.
- Schedule robot and task IDs must reference records that passed validation in their respective files.

Feasibility Logic

- A task is executable if at least one robot is in the same zone as both the source and target destinations and can carry the package weight.
- A schedule is feasible if the robot can carry every package, all destinations are in the robot's zone, and the robot can return to the origin without the battery reaching 0% mid-journey.
- Battery drains at 1% per km when travelling empty, with an additional 0.5% per kg per km when carrying a package.
- Robots travel at a uniform speed of 15 km/h. Travel time and distance are computed from the distance matrix.

Output

- Write a feasibility_report.txt listing each task as executable or not executable, with totals, followed by a schedule section showing time, distance, and battery for each feasible schedule.
- Save a matplotlib plot showing each feasible robot's distance from the origin over time, with each schedule as a labelled line.

Technologies Used

- **Language:** Python 3
- **Modules:** re, csv, sys, pandas, matplotlib
- **Architecture:** Separated into reader.py (data loading and validation), tasker.py (feasibility logic), and main.py (output and entry point)